



TIPS & TRICKS



How to display the I/O statistics for the `dd` command

When we use the `dd` command to convert and copy files, by default, the I/O statistics are not shown. But sometimes it is important to print I/O statistics to view the progress of the command. To do so, on the standard error stream, send a USR1 signal to the running `dd` command. Let us look at an example.

As the `dd` command comes by default with most of the GNU/Linux distributions, there is no need to install it separately. So let us proceed with opening a terminal and executing the `dd` command, as follows:

```
[bash]$ dd if=/dev/zero of=/dev/null
```

Open one more terminal, find the PID of the `dd` command and send the USR1 signal to it as shown below:

```
[bash]$ pgrep dd
3989
```

```
[bash]$ kill -USR1 3989
```

The `Kill` command will return immediately without printing anything. Now, move to the first terminal where the `dd` command was running and you will find the I/O statistics are printed on the standard error stream. After printing the I/O statistics, the `dd` command will resume its copy operation.

Given below is a sample output of I/O statistics:

```
786328388+0 records in
```

```
786328387+0 records out
```

```
402600134144 bytes (403 GB) copied, 251.012 s, 1.6 GB/s
```

—Narendra Kangralkar,
narendrakangralkar@gmail.com



Find and remove files with one command, on-the-fly

Sometimes it is necessary to find out files and remove them. However, the `rm` command does not support search

criteria. You need to use the `find` command to search for files in a directory and remove them on-the-fly.

Here is an example to demonstrate how we can search and delete files:

```
#touch need_to_delete.txt
#ls -l need_to_delete.txt
-rw-rw-r-- 1 tsg tsg 0 Oct 28 22:34 need_to_delete.txt

#find . -name "*need_to*"
./need_to_delete.txt
```

To delete the file, add `-delete` at the end of the `find` command:

```
#find . -name "*need_to*" -delete
#ls -l need_to_delete.txt
```

```
ls: cannot access need_to_delete.txt: There is no such file
or directory
```

If the removal failed, an error message is issued. If `'-delete'` fails, the exit status of `'find'` will be non-zero (when it eventually exits). The use of `'-delete'` automatically turns on the `'-depth'` option.

Warning: Don't forget that the `'find'` command line is evaluated as an expression; so if you put `'-delete'` first, `'find'` will try to delete everything below the starting points you specified. When testing a `'find'` command line that you later intend to use with `'-delete'`, you should explicitly specify `'-depth'` in order to avoid later surprises. Because `'-delete'` implies `'-depth'`, you cannot effectively use `'-prune'` and `'-delete'` together.

—Mohamed Mansoor,
mohamedmansoor.anvar@matrimony.com



Archive files using `'ar'`

Here is a Linux command that is not very common. `ar` is used to archive files. Shown below are a few of the options and details used with `ar`:

```
ar -q archive-file file
```